

Design and Hardware Implementation of a 16-Tap FIR Filter on FPGA: An Evaluation of Area and Throughput Trade-offs

Nguyen Pham Gia Bao¹, Nguyen Manh Cuong¹, Nguyen Nhat Huy¹, Nguyen Quang Huy¹, Ho Hoang Dung¹

¹ Semiconductor IC Design Major, FPT University, Ho Chi Minh City, Vietnam

Abstract. This study presents the design methodology and performance evaluation of a 16-tap Finite Impulse Response (FIR) filter implemented on a Field-Programmable Gate Array (FPGA) platform. The primary objective of this project is to address the classical trade-off problem in digital integrated circuit design by developing two specialized hardware architectures: the Folded architecture (area optimization) and the Pipelined architecture (throughput optimization). Coefficient data is standardized into the Q15 fixed-point integer format to accommodate hardware resource constraints. Verification results on ModelSim and synthesis results from Vivado/Quartus software indicate that the Folded architecture maximizes Digital Signal Processing (DSP) block savings (utilizing only 1 block), whereas the Pipelined architecture achieves a throughput of 1 sample/cycle with a maximum operating frequency (F_{max}) reaching 211.55 MHz.

Keywords: FPGA, FIR Filter, Folded MAC, Pipelined Architecture, DSP, Fixed-point Arithmetic.

1 Introduction

In modern digital signal processing (DSP) and telecommunication systems, FIR filters play a fundamental role in tasks such as noise filtering, anti-aliasing prior to modulation, and pulse shaping [1, 4]. FIR filters are preferentially implemented on FPGAs due to their finite impulse response capability, absolute stability (absence of feedback loops), and robust support for parallelization or Pipelining techniques [2].

However, to achieve an ideal filter response, the filter requires a large number of taps, leading to an increase in multiplications and significant consumption of hardware resources (DSPs, LUTs, Registers). This paper proposes an RTL design optimization solution at the system level by developing Folded and Pipelined architectures in parallel, providing an empirical database for selecting the appropriate architecture depending on practical applications.

2 Theoretical Background

2.1 Mathematical Equation of the FIR Filter

The mathematical essence of an FIR filter is the linear convolution equation, defined as follows [1]:

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

Where $y[n]$ is the output signal, $x[n]$ is the input signal, $h[k]$ is the filter coefficient array, and N is the number of filter taps.

2.2 Fixed-point Arithmetic

Hardware implementation on FPGAs requires converting the coefficients $h[k]$ from floating-point numbers to signed integers (Fixed-point) to reduce the complexity of logic and DSP blocks. The conversion process utilizes the Q-format (specifically Q1.15) with the standardizing equation [2]:

$$h_{fixed} = \text{round}(h_{float} \times 2^F)$$

With $F = 15$, decimal values are represented as primitive integers (e.g., a coefficient of 0.1254 corresponds to the value 4109 in decimal or 100D in Hexadecimal format).

2.3 Mathematical Equation of the FIR Filter

The mathematical essence of an FIR filter is the linear convolution equation, defined as follows [1]:

$$y_n = \sum_{k=0}^{N-1} h_k x_{n-k}$$

Where y_n is the output signal, x_n is the input signal, h_k is the filter coefficient array, and N is the number of filter taps. Prior to hardware implementation, the ideal floating-point coefficients (h_{float}) were synthesized using the Windowing method (e.g., Hamming window) via Python-based DSP libraries. This algorithmic approach ensures the filter meets the specific low-pass frequency characteristics, including the desired cut-off frequency and stopband attenuation, before undergoing quantization.

Fixed-point Arithmetic

Hardware implementation on FPGAs requires converting the coefficients h_k from floating-point numbers to signed integers (Fixed-point) to reduce the complexity of logic and DSP blocks...

Table 1. Coefficient Conversion Example (Q15 Format)

Coefficient	Float Value	Fixed-point (Decimal)
H[0]	0.1254	4109
H[1]	-0.0521	-1707
Coefficient	Float Value	Hexadecimal
H[0]	0.1254	100D
H[1]	-0.0521	F955

3 Proposed Hardware Architectures

3.1 Symmetric FIR Optimization (Linear Phase)

To optimize hardware from the algorithmic level, the architecture leverages the symmetric property of linear-phase FIR filters ($h[k] = h[N - 1 - k]$). The convolution equation is restructured as follows [4]:

$$y[n] = \sum_{k=0}^{\frac{N}{2}-1} h[k](x[n-k] + x[n-(N-1-k)])$$

This technique allows the pairing of data samples sharing the same coefficient through a Pre-adder before executing the multiplication, thereby reducing the number of Multipliers by 50%.

3.2 Area Optimization (Folded MAC Architecture)

This design is targeted at systems with stringent resource constraints. The Data Path structure is Folded, utilizing exactly 1 Multiplier and 1 Adder. A Finite State Machine (Control FSM) and Multiplexers (MUX) are employed to coordinate iterative computations over multiple clock cycles.

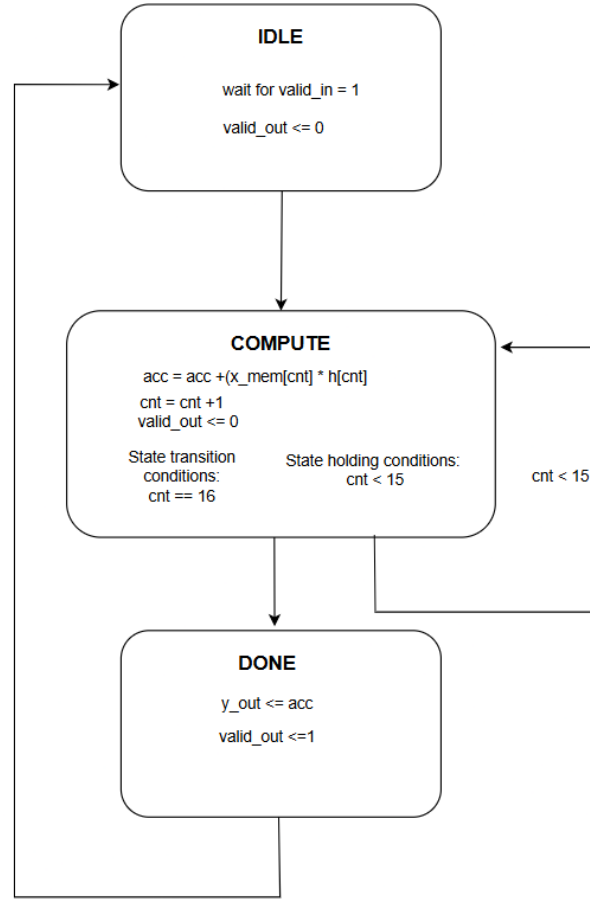


Fig. 1 Finite State Machine (FSM) of the proposed Folded MAC architecture.

This state machine acts as the control center for our iterative filtering process. It rests in the IDLE state until a valid input arrives. During the COMPUTE phase, an internal counter loops through 16 clock cycles, directing the single MAC unit to process all filter taps sequentially. Finally, the DONE state safely latches the accumulated result to the output before resetting the system for the next input.

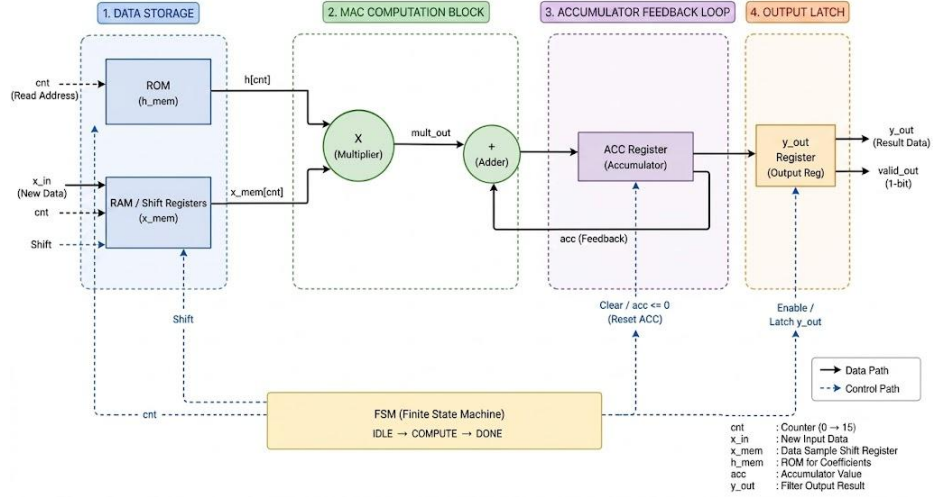


Fig. 2 Data path of the proposed Folded MAC architecture.

Designed to minimize hardware resources, this architecture cleverly reuses a single multiplier and adder pair. Under the FSM's guidance, incoming data and fixed coefficients are fed step-by-step into the computing block. The accumulator maintains a running total over multiple cycles, effectively trading processing time for a significantly smaller silicon footprint.

3.3 Throughput Optimization (Pipelined Architecture)

This design focuses on maximizing the volume of data processed per second. By inserting Pipeline registers between combinational logic blocks, the Critical Path is significantly shortened [3]. The Data Path implements an array of 8 Multipliers operating in parallel, combined with an Adder Tree structure.

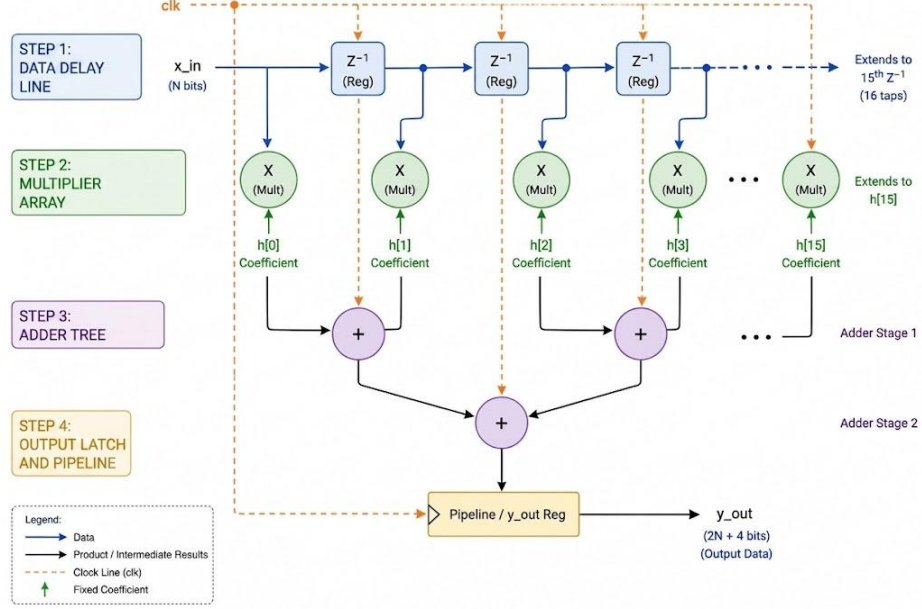


Fig. 3. Hardware architecture of the Pipelined FIR filter demonstrating the Pre-adder and parallel multiplier array.

Built for speed, this design is capable of processing a new sample every single clock cycle. It takes advantage of the filter's symmetric coefficients to cut the required multipliers in half. By strategically placing pipeline registers across the multiplier array and adder tree, we effectively break the critical path, allowing the FPGA to achieve much higher operating frequencies.

3.4 Bit-width Planning

To prevent overflow, bit-widths are systematically allocated across the data path. Given that both the input data x_{in} and the coefficients h_k are 16-bit signed integers, their multiplication yields a 32-bit product. For a 16-tap FIR filter ($N = 16$), the accumulator requires $\lceil \log_2(N) \rceil = 4$ guard bits to prevent overflow during summation. Consequently, the final output y_{out} is architected at $2N + 4$ bits (i.e., 36-bit) to maintain full precision without data loss.

4 Simulation and Synthesis Results

4.1 Functional Verification and Waveform Analysis

A comprehensive testbench environment was constructed to inject input signals corrupted with high-frequency noise. The Folded architecture requires 17 clock cycles per sample (including MAC cycles and FSM transition overhead), whereas the Pipelined architecture delivers a steady throughput of 1 sample per clock cycle after the initial pipeline latency.

Spectral Analysis (FFT): To verify functional accuracy, output data was extracted from ModelSim and processed using MATLAB for spectral analysis. As shown in Fig. 4, the signal spectrum confirms that high-frequency noise is effectively suppressed by both designs. The spectral responses of the two architectures exhibit a negligible deviation, with a maximum magnitude difference of 0.0000 dB, confirming the functional equivalence of the RTL implementations.

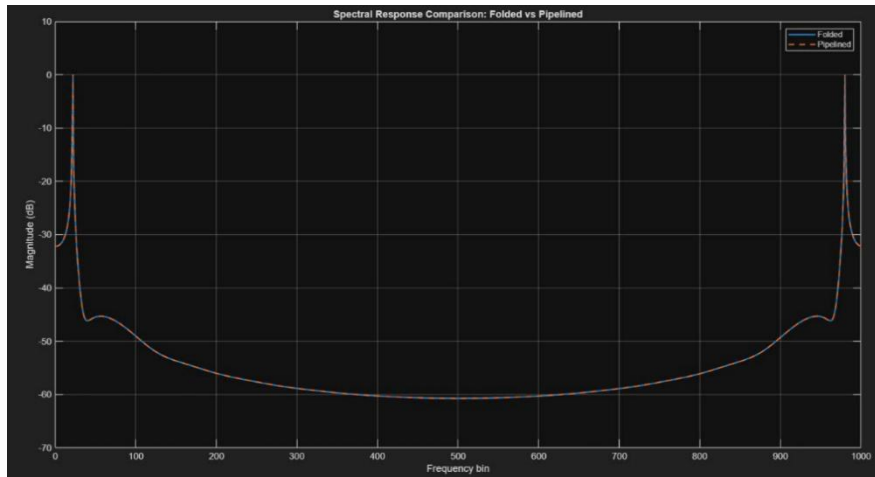


Fig. 4. Spectral response comparison generated by MATLAB, confirming a 0.0000 dB deviation between the Folded and Pipelined architectures.

4.2 Resource and Timing Analysis

The following table summarizes the synthesis results obtained using Vivado [5].

Table 2: Synthesis Comparison Summary

Metric	Folded Architecture	Pipelined Architecture
DSP Blocks	1	8
Slice LUTs	109	366

F_{max}	118.83 MHz	211.55 MHz
Throughput	1 sample / 17 cycles*	1 sample / 1 cycle

The Folded architecture is governed by a Finite State Machine (FSM) that explicitly requires 17 clock cycles to process a single data sample. This latency comprises 16 clock cycles in the COMPUTE state—iteratively calculating the Multiply-Accumulate (MAC) operations for all 16 taps—and 1 additional cycle in the DONE state to latch the final accumulator value to the output register. Consequently, its throughput is strictly limited to 1 sample per 17 cycles. In contrast, the Pipelined architecture leverages a parallel multiplier array and deep pipeline registers. Once the initial pipeline stages are filled, it sustains a maximum throughput of 1 output sample per clock cycle, making it vastly superior for high-bandwidth applications.

4.3 Trade-off Analysis

The study confirms the inversely proportional relationship between area and throughput. The Folded architecture is an ideal solution for low-cost IoT devices or embedded systems with restricted power and area. Conversely, the Pipelined architecture consumes more resources but is a mandatory requirement for broadband applications such as 5G telecommunication base stations that require processing continuous data streams.

5 Conclusion

This project successfully designed, implemented at the RTL level using Verilog, and rigorously verified two distinct 16-tap FIR filter architectures utilizing fixed-point arithmetic. The comparative research not only effectively addresses fundamental IC integration challenges—specifically MAC architecture optimization and timing closure—but also establishes a valuable empirical database demonstrating area-throughput trade-offs. Future work will focus on integrating Polyphase Filter techniques and sampling rate conversion structures (Interpolation and Decimation Filters) to further enhance energy efficiency and hardware utilization for radio transceiver IP cores.

References

- [1] Proakis, J. G., Manolakis, D. G.: Digital Signal Processing: Principles, Algorithms, and Applications. 4th edn. Pearson Prentice Hall, Upper Saddle River (2006).
- [2] Meyer-Baese, U.: Digital Signal Processing with Field Programmable Gate Arrays. 4th edn. Springer-Verlag, Berlin Heidelberg (2014).
- [3] Parhi, K. K.: VLSI Digital Signal Processing Systems: Design and Implementation. John Wiley & Sons, New York (1999).
- [4] Oppenheim, A. V., Schaffer, R. W.: Discrete-Time Signal Processing. 3rd edn. Pearson, London (2009).

[5] Xilinx, Inc.: Vivado Design Suite User Guide: Synthesis (UG901). San Jose, CA (2020).